

GCPO: Diagnosing and Constraining Subspace Geometry in Rollout RL for LLMs

Kai Yang^{1,2,3†}, Jingwei Xu^{3†}, Wanyu Wang⁴, Kai-Yuan Guo⁵, Zhenbo Yu^{3,6✉}, Yi Wang^{2✉*}, Yu Qiao^{1,2✉}

¹School of Computer Science, Shanghai Jiao Tong University

²Shanghai Artificial Intelligence Laboratory

³NovaCore

⁴SJTU Paris Elite Institute of Technology, Shanghai Jiao Tong University

⁵School of Artificial Intelligence and Automation, Huazhong University of Science and Technology

⁶School of Artificial Intelligence, Shanghai Jiao Tong University

{icarus1411, wwyspeit}@sjtu.edu.cn, {jingwei.xu, zhenbo.yu}@deepnovacore.com, gky@hust.edu.cn, wygamle@gmail.com, qiaoyu@pjlab.org.cn

Abstract

On-policy rollout methods such as GRPO are central to post-training of large language models. Yet, they frequently suffer from training instabilities, cross-task capability degradation, and response-length inflation. Although prior work has characterized the subspace geometry of aggregate updates, the stepwise variation of this geometry and its relationship to model performance remain unclear. We introduce *Principal-Subspace Overlap*, a dimension-corrected measure of individual rollout updates relative to the dominant singular subspaces of pretrained weights. Despite low average overlap, transient spikes often precede performance degradation. To address this, we propose **GCPO** (Geometrically Constrained Policy Optimization), which applies hard bilateral orthogonal projections to constrain updates to the complementary subspaces, preventing such excursions by construction. Across mathematical reasoning, code generation, and tool-use tasks on Qwen3-8B and GLM4-9B, GCPO consistently outperforms GRPO and recent variants, including DAPO and GSPO, improving over the base models and the strongest baseline by up to 27.69 and 2.37 points, respectively. Furthermore, GCPO preserves general capabilities, eliminates response-length inflation, and stabilizes policy entropy. Our findings provide a new diagnostic lens and a principled design perspective for stable reinforcement learning post-training.

Code & Datasets — <https://github.com/Icarus1411/GCPO>

1 Introduction

Rollout-based reinforcement learning (RL) has become a widely used post-training mechanism for improving large language models (LLMs) on mathematical reasoning, coding, and tool use (Schulman et al. 2017; Wang et al. 2022; Gao, Schulman, and Hilton 2023). Its optimization data, however, are generated by the evolving policy itself: each update changes both the policy and the rollout distribution used

to construct the next update. This feedback loop can yield unstable training, degradation on capabilities outside the optimized task, and response-length inflation (Harmon et al. 2025; Ouyang et al. 2022).

Most existing remedies stabilize RL through objective- or policy-level controls. KL regularization constrains divergence from a reference policy, clipping limits large likelihood-ratio changes, and reward design modifies the optimization signal (Zheng et al. 2025; Yu et al. 2026b; Zhao et al. 2025). While effective, these methods may treat different updates as similar if they have similar policy-level statistics, even when their directions in parameter space differ. Because updates of similar magnitude may interact differently with the structured parameter space of the pretrained model, we ask whether their stepwise directions provide a complementary diagnostic of training dynamics.

We focus on the dominant singular subspaces of each pretrained linear weight matrix. The top right and left singular vectors identify the dominant input and output directions, respectively. Together, they define the optimal rank- k approximation of the pretrained transformation (Golub, Hoffman, and Stewart 1987). Updates overlapping these subspaces can therefore alter structurally prominent components of the pretrained transformation. Rather than interpreting these directions as a literal decomposition of semantic knowledge, we treat them as a functionally distinguished yet tractable structural reference. Prior analyses further show that RL updates are predominantly off-principal when averaged over training (Shen et al. 2026; Yu et al. 2026a; Schneider et al. 2024). This aggregate regularity leaves open an important question: do individual updates exhibit transient principal-subspace overlap, and how is such overlap related to training performance?

Our stepwise analysis reveals transient entry of individual updates into the pretrained principal subspaces. We decompose each realized update into four blocks according to its left- and right-side overlap with these subspaces, and subtract the overlap expected under an isotropic null. Although most update energy lies in the doubly orthogonal block, aggregate statistics obscure intermittent spikes in *excess principal-*

[†]These authors contributed equally.

[✉]Corresponding author.

*Contributions made while affiliated with Shanghai AI Lab.

subspace overlap. Across the runs we examine, sustained or repeated spikes coincide with, and often precede, declines in validation accuracy. We therefore treat overlap as a geometric correlate of unstable phases, motivating updates that preserve the selected pretrained mappings by remaining in their bilateral orthogonal complement.

We instantiate this hypothesis in **GCPO** (Geometrically Constrained Policy Optimization), which augments rollout-based policy optimization with a hard geometric constraint on the effective policy update. At each rollout iteration, GCPO optimizes the underlying policy objective while requiring every adapted layer update to lie in the bilateral orthogonal complement of the selected pretrained principal subspaces. GCPO thus changes the admissible directions of policy improvement, rather than introducing another policy-level penalty. The constraint is complementary to KL regularization: KL controls policy change in output space, whereas GCPO controls the feasible directions of parameter change. It also has a precise layer-level guarantee: for inputs in a selected principal input subspace, the adapted layer’s response is unchanged. The remaining complement is large, retaining a high-dimensional feasible complement for task adaptation.

Comprehensive evaluations on Qwen3-8B and GLM4-9B cover mathematical reasoning, code generation, and tool use. Across all tasks, GCPO achieves the best accuracy, outperforming the strongest baseline by 1.02–2.37 points ($\pm$ std over 3 seeds) and the corresponding instruction-tuned model by 7.09–27.69 points. When trained on mathematics and evaluated on other tasks, it attains the strongest worst-case retention. It also shows smoother accuracy and policy-entropy trajectories and suppresses response-length inflation.

In summary, our main contributions are threefold:

- We introduce a dimension-corrected, stepwise measure of principal-subspace overlap and show that elevated overlap is a warning signal associated with declining validation performance in rollout RL.
- We formulate rollout RL as a constrained policy optimization problem and propose GCPO to enforce bilateral orthogonality on every effective policy update, which preserves the principal pretrained mappings while retaining a large complementary update space.
- Across two model families and three task domains, GCPO outperforms all evaluated baselines, better preserves cross-task capabilities, and exhibits more stable accuracy, policy-entropy, and response-length dynamics.

2 Related Work

Rollout-based RL for LLM Alignment. Rollout-based reinforcement learning has emerged as the dominant paradigm for enhancing LLM reasoning capabilities. Building upon PPO (Schulman et al. 2017), recent advances like GRPO (Shao et al. 2024), GSPO (Zheng et al. 2025), GMPO (Zhao et al. 2025), and DAPO (Yu et al. 2026b) have significantly improved advantage estimation and scaling efficiency. Despite these innovations, the dynamic nature of self-generated rollouts frequently drives policies toward optimization instabilities, response-length inflation, and general capability degradation (Yang et al. 2025). Crucially, existing

methods primarily intervene through objectives or observable policy behavior, leaving the geometry of realized parameter updates less explored. This highlights the need to diagnose the roots of these instabilities from a novel scope rather than symptomatic output-space patching.

Regularization and Stability in RL. RL post-training is commonly stabilized through objective- or policy-level controls. KL penalties discourage deviation from a reference policy, clipping constrains large likelihood-ratio changes, and reward shaping adjusts the scalar optimization signal to reduce undesirable behaviors such as reward hacking or length inflation (Singhal et al. 2023; Li et al. 2026). These techniques are effective, but they remain soft controls: they discourage unstable updates rather than ruling them out. Their effectiveness often depends on carefully tuned penalty coefficients and can be weakened by the high-variance feedback of rollout-based RL (Achiam et al. 2017; Grontas et al. 2025). These limitations motivate looking beyond objective-level controls and designing a complementary approach that constrains the feasible update space directly.

Geometric Analysis of Policy Optimization. Recent studies suggest that the geometry of parameter updates plays a critical role in RL-based post-training. Rather than treating policy updates as unstructured perturbations, these studies show that RL-induced updates exhibit systematic geometric patterns relative to the pretrained weights. In particular, recent analyses find that successful RL updates are, on average, more concentrated outside the dominant singular subspaces of the pretrained operators (Shen et al. 2026; Cai et al. 2026). This off-principal tendency suggests that effective policy adaptation may preferentially exploit directions that interfere less with the dominant structures learned during pre-training, which provides a foundation for understanding the importance of parameter-space geometry in rollout-based RL. However, existing analyses mainly characterize aggregate update behavior, leaving open how stepwise deviations from this geometry emerge during training and whether they are related to optimization instability.

3 Principal-Subspace Overlap and Instability

Prior work finds that RL updates, when aggregated over training, predominantly lie outside the dominant singular subspaces of pretrained weights (Shen et al. 2026; Cai et al. 2026). Nevertheless, aggregate statistics may obscure intermittent spikes, termed *excess principal-subspace overlap*, which often precede validation degradation. Controlled interventions further show that amplifying this overlap causes dose-dependent accuracy drops, motivating complementary-subspace constraints.

3.1 Preliminaries

For a pretrained weight matrix $W_{ref} \in \mathbb{R}^{d_{out} \times d_{in}}$, let $W_{ref} = \Phi \Sigma \Psi^\top$ be its SVD. We call the spans of the top- k left and right singular vectors, Φ_k and Ψ_k , its *principal subspaces*. They capture high-energy input–output directions of the pretrained linear operator; they are a structural proxy, not a literal partition of semantic knowledge. Let

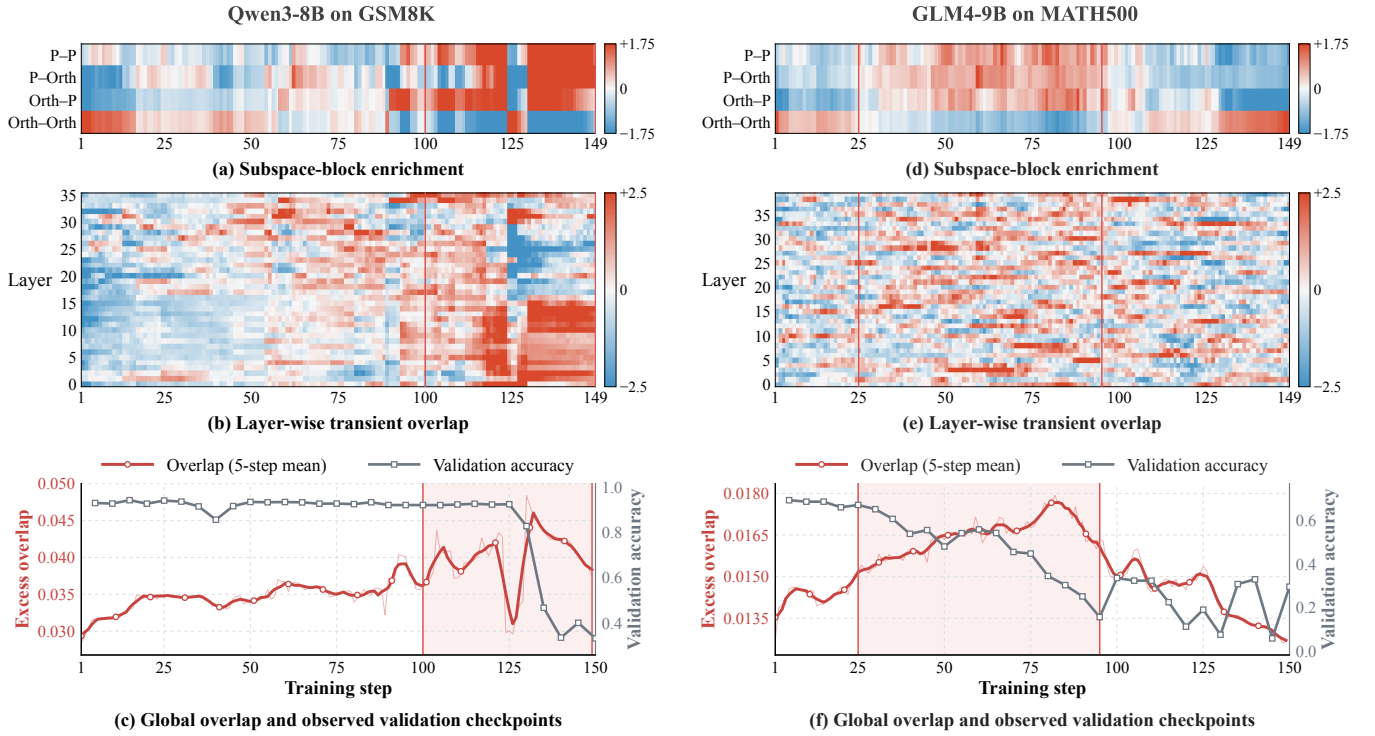


Figure 1: Stepwise update overlap and validation performance. The red curve is the 5-step moving average of excess principal-subspace overlap; the blue curve is validation accuracy. In both runs, episodes of elevated overlap accompany subsequent gradual degradation. This observation is correlational.

$\Pi_\Phi = \Phi_k \Phi_k^\top$, $\Pi_\Psi = \Psi_k \Psi_k^\top$ be the associated projectors, and $\Pi_\Phi^\perp = I - \Pi_\Phi$, $\Pi_\Psi^\perp = I - \Pi_\Psi$ denote the projectors onto their orthogonal complements.

We analyze the realized update $\delta^{(t)}W = W_t - W_{t-1}$, rather than the raw gradient, so the measurement includes the optimizer, learning rate, and momentum. Its energy can be decomposed into four mutually orthogonal blocks:

$$\begin{aligned} \delta^{(t)}W &= \underbrace{\Pi_\Phi \delta^{(t)}W \Pi_\Psi}_{\delta^{(t)}W^{PP}} + \underbrace{\Pi_\Phi \delta^{(t)}W \Pi_\Psi^\perp}_{\delta^{(t)}W^{PO}} \\ &+ \underbrace{\Pi_\Phi^\perp \delta^{(t)}W \Pi_\Psi}_{\delta^{(t)}W^{OP}} + \underbrace{\Pi_\Phi^\perp \delta^{(t)}W \Pi_\Psi^\perp}_{\delta^{(t)}W^{OO}}. \end{aligned} \quad (1)$$

Let $E_{ij} = \|\delta^{(t)}W^{ij}\|_F^2$ and $E_{\text{total}} = \|\delta^{(t)}W\|_F^2$. The OO block is orthogonal to the principal subspace on both sides; the other three blocks have overlap on at least one side. We summarize this overlap by

$$O_t = \frac{E_{PP} + E_{PO} + E_{OP}}{E_{\text{total}}} = 1 - \frac{E_{OO}}{E_{\text{total}}}. \quad (2)$$

3.2 Observed Excess Principal-subspace Overlap

This raw ratio depends partly on dimensionality: an isotropic update has nonzero overlap simply because the principal subspaces have dimension k . For a $d_{\text{out}} \times d_{\text{in}}$ matrix, its expected overlap is

$$O_{\text{null}} = 1 - \frac{(d_{\text{out}} - k)(d_{\text{in}} - k)}{d_{\text{out}} d_{\text{in}}}. \quad (3)$$

We therefore report the dimension-corrected quantity

$$O_t^{\text{excess}} = O_t - O_{\text{null}}. \quad (4)$$

Positive O_t^{excess} means that an update is more aligned with the principal subspaces than an isotropic update of the same size. We refer to this quantity as *excess principal-subspace overlap*. It is an observable alignment statistic, not a claim that every principal direction is harmful.

Figure 1 shows two consistent findings. First, most update energy remains in OO (93.8% for Qwen3-8B and 97.7% for GLM4-9B on average), agreeing with the aggregate off-principal trend. Second, this average masks intermittent positive spikes in O_t^{excess} . In the displayed runs, sustained or repeated spikes coincide with, and precede, declining validation accuracy; the effect appears as gradual deterioration in one setting and a sharp drop in another. Layer-wise measurements further show that these spikes are concentrated rather than uniform, often in intermediate and upper layers. More results are provided in Appendix E.

Controlled intervention. To move beyond correlation, we intervene on a GRPO update of Qwen3-8B on ToolAlpaca. We rescale its principal-overlapping component by η while preserving the Frobenius norm of every layer-wise update. Details are provided in Appendix D. As shown in Figure 2, increasing overlap produces a clear dose-dependent accuracy drop. Relative to the original update ($\eta = 1$, 56.89%), removing the overlapping component improves accuracy, whereas principal-subspace injection substantially reduces accuracy.

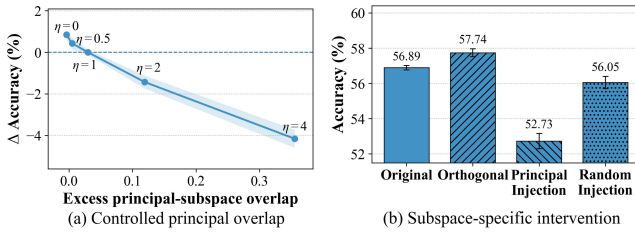


Figure 2: Controlled intervention on the step-150 GRPO update of Qwen3-8B on ToolAlpaca. (a) Increasing the principal-overlapping component under layer-wise norm matching produces a dose-dependent accuracy drop. (b) Orthogonalization improves accuracy, whereas matched principal-subspace injection is substantially more harmful than a random-subspace intervention.

A matched random-subspace intervention causes substantially less degradation. These results provide local intervention evidence that increasing principal-subspace overlap can directly harm model performance, beyond the effects of update magnitude or arbitrary directional perturbation. Additional interventions across models, tasks, and checkpoints show the same dose-dependent trend (Appendix E).

Together, the observational and intervention results motivate preserving the dominant input and output directions while optimizing in their bilateral orthogonal complement. We implement this restriction next.

4 Methodology

Our diagnostic suggests a simple intervention: rather than modifying the rollout objective, restrict each policy update to the bilateral orthogonal complement of the dominant singular subspaces of the pretrained weights. GCPO realizes this intervention within GRPO through a projected low-rank parameterization.

4.1 Problem Formulation

Let θ_0 be the pretrained policy and $\mathcal{M}$ the set of adapted linear layers. For each $\ell \in \mathcal{M}$, we compute once the rank- k singular subspaces $\Phi_k^{(\ell)}$ and $\Psi_k^{(\ell)}$ of $W_{\text{ref}}^{(\ell)}$. We seek a policy update that maximizes the usual rollout objective while having zero principal-subspace overlap at every adapted layer.

$$\max_{\Delta\theta} \mathcal{J}_{\text{rollout}}(\pi_{\theta_0+\Delta\theta}) - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta_0+\Delta\theta} \parallel \pi_{\text{ref}}), \quad (5)$$

$$\text{s.t.} \quad \Phi_k^{(\ell)\top} \delta^{(t)} W^{(\ell)} = 0, \quad (6)$$

$$\delta^{(t)} W^{(\ell)} \Psi_k^{(\ell)} = 0, \quad \forall \ell \in \mathcal{M}, \quad (7)$$

The two constraints are equivalent to retaining only the OO block of each update. They complement, rather than replace, KL regularization: KL controls policy change in output space, whereas the constraints control where the parameter update can lie. A penalty could encourage these constraints, but would require trading them against the rollout objective. We instead enforce them exactly by parameterization.

4.2 GCPO: Geometrically Constrained Policy Optimization

For each adapted layer, with fixed orthogonal-complement projectors $\Pi_{\Phi}^{\perp}, \Pi_{\Psi}^{\perp}$, we can parameterize its update as

$$\delta^{(t)} W^{(\ell)} = \alpha \Pi_{\Phi}^{\perp} L^{(\ell)} R^{(\ell)} \Pi_{\Psi}^{\perp}, \quad (8)$$

where $L^{(\ell)}$ and $R^{(\ell)}$ are trainable low-rank factors and α is the standard scaling constant. The right projector removes components acting on dominant input directions; the left projector removes components written into dominant output directions.

This parameterization satisfies both constraints for every value of the trainable factors in Eq. (6) and Eq. (7). Thus, even if rollout gradients contain principal-subspace components, those components cannot change the effective layer update. This is the central property of GCPO: it converts the observed preference for off-principal updates into an exact feasible set, while leaving optimization within that set unchanged.

4.3 Theoretical Properties and Comparison

Exact subspace preservation. GCPO satisfies the bilateral constraints at every adapted layer and every optimization step. Consequently, the pretrained layer mapping is unchanged for inputs in the selected principal input subspace, with an analogous guarantee for the principal output subspace. Formal proofs are provided in Appendix B.

Capacity and cost. The feasible update space has dimension $(d_{\text{out}} - k)(d_{\text{in}} - k)$, and every feasible rank- r update can be represented by the projected low-rank parameterization. Thus, for k small relative to the layer width, GCPO retains a large space for task adaptation. Formal proofs are provided in Appendix B.

Distinction from Low-Rank Adaptation. Although GCPO uses low-rank factorization for efficient implementation, it solves a different constrained optimization problem from conventional low-rank adaptation. LoRA primarily restricts update rank for parameter efficiency, while some geometric variants shape update directions to prevent catastrophic forgetting typically under fixed-data supervised fine-tuning (Schotthöfer et al. 2024; Wang et al. 2025; Dragoi et al. 2026; Wang et al. 2023). In contrast, GCPO is motivated by transient *principal-subspace overlap* under on-policy rollout feedback in RL, where each policy update changes the subsequent training distribution. It therefore constrains update direction through bilateral orthogonality to stabilize stepwise RL dynamics and preserve pretrained mappings, while low rank serves mainly as an efficient parameterization of this feasible space.

5 Experiments

In this section, we evaluate GCPO in terms of general-task performance, cross-task capability retention, training stability and efficiency, and design effectiveness through ablation and structural analyses.

Models and Tasks. We adopt two widely used instruction-tuned LLMs as our backbone models: Qwen3-8B (Yang et al. 2025) and GLM4-9B (GLM et al. 2024), and evaluate them across three representative domains using established benchmarks and consistent protocols across methods:

- **Mathematical Reasoning:** Evaluated on the MATH500 (Lightman et al. 2024). The reward is computed via exact matching of the final boxed answer using rule-based parsing.
- **Code Generation:** Evaluated on HumanEval+ (Liu et al. 2023) for Python function synthesis. The reward is granted based on the execution pass rate across unit tests.
- **Tool Use:** Evaluated on ToolAlpaca (Tang et al. 2023), which requires generating the appropriate API call given a tool specification and a user request. The reward is based on exact function-name matching, argument-key consistency, and normalized argument-value matching.

Baselines. We extensively compare GCPO against a comprehensive suite of optimization strategies. These include GRPO (Shao et al. 2024), objective-level variants including GSPO (Zheng et al. 2025), DAPO (Yu et al. 2026b), and GMPO (Zhao et al. 2025), which modify objectives or clip probability ratios. To compare with parameter-space constraints, we include GRPO-LoRA (Hu et al. 2022), which uses unconstrained low-rank adaptation without the hard bilateral orthogonality enforced by GCPO. To isolate the effect of the directional constraint from low-rank parameterization, GRPO-LoRA and GCPO use the same adaptation rank and scaling configuration.

Implementation Details. For all GRPO-based methods, we sample $K = 16$ rollouts per prompt during training. We adopt the same hyperparameters to ensure equity. For GCPO, we precompute the top- $k = 8$ singular subspaces and constrain updates to their bilateral orthogonal complements. For evaluation, we report bootstrap-estimated majority@16 accuracy from 16 test-time responses per example. More details on seeds, data splits, evaluation, and hyperparameters are provided in Appendix D.

5.1 Main Results

Table 1 reports the mean and standard deviation over three independent training seeds across three task domains and two backbone models. GCPO achieves the best accuracy in all six model-task settings. Relative to the corresponding instruction-tuned base models, it improves accuracy by 7.09–27.69 percentage points, and surpasses the strongest competing method on each benchmark by 1.02–2.37 points. Moreover, GCPO exhibits the lowest standard deviation in all six settings, indicating reduced sensitivity to training stochasticity. The margins are particularly pronounced on GLM4-9B, where GCPO outperforms the strongest baselines by 2.15–2.37 points, demonstrating consistent effectiveness across backbones with different optimization behaviors.

Beyond individual benchmarks, GCPO also achieves the strongest average performance. On Qwen3-8B, it obtains an average accuracy of 78.63, exceeding the strongest baseline average of 77.48 by 1.15 points. On GLM4-9B, it im-

proves the strongest baseline average from 73.73 to 76.12. Averaged across all six model-task settings, GCPO reaches 77.38, compared with 75.61 for the strongest baseline, which suggests effectiveness and generalizability.

Crucially, we emphasize that these gains are not merely artifacts of low-rank parameter reduction: as shown in Table 1, GRPO-LoRA with the same rank achieves consistently lower accuracy than GCPO across all evaluations. This confirms that the explicit directional orthogonal projection is the primary driver of our method’s superiority.

Method	MATH500	HumanEval+	ToolAlpaca
<i>Qwen3-8B</i>			
Base (Instruct)	67.46	73.58	56.53
GRPO	72.00 $\pm$ 1.36	84.24 $\pm$ 0.88	59.56 $\pm$ 1.52
GSPO	77.80 $\pm$ 0.74	87.81 $\pm$ 0.51	66.18 $\pm$ 0.82
DAPO	78.33 $\pm$ 0.61	88.13 $\pm$ 0.48	65.99 $\pm$ 0.76
GMPO	77.64 $\pm$ 0.69	88.14 $\pm$ 0.45	66.16 $\pm$ 0.71
GRPO-LoRA	77.87 $\pm$ 0.58	87.36 $\pm$ 0.62	66.05 $\pm$ 0.67
GCPO	79.47 $\pm$0.31	89.16 $\pm$0.27	67.26 $\pm$0.39
<i>GLM4-9B</i>			
Base (Instruct)	66.51	76.55	42.47
GRPO	59.43 $\pm$ 1.84	72.43 $\pm$ 1.42	65.70 $\pm$ 1.65
GSPO	71.29 $\pm$ 0.97	79.55 $\pm$ 0.83	66.22 $\pm$ 1.08
DAPO	72.41 $\pm$ 0.82	81.43 $\pm$ 0.71	67.35 $\pm$ 0.93
GMPO	71.33 $\pm$ 0.91	80.63 $\pm$ 0.76	67.79 $\pm$ 0.88
GRPO-LoRA	72.34 $\pm$ 0.66	81.48 $\pm$ 0.59	66.19 $\pm$ 0.75
GCPO	74.56 $\pm$0.34	83.64 $\pm$0.29	70.16 $\pm$0.41

Table 1: Main results across tasks. We report bootstrap-estimated majority@16 accuracy as mean $\pm$ standard deviation over three training seeds, in percentage points.

5.2 Cross-Task Capability Preservation

Single-domain RL post-training may improve the target task while degrading other capabilities. To examine whether our geometric constraint better preserves such general capabilities, we post-train models on MATH500 and evaluate their cross-task capabilities on code generation (HumanEval+) and tool-use (ToolAlpaca) benchmarks.

Existing RL variants poorly retain tool-use capability.

As shown in Table 2, GRPO and its variants generally exhibit substantial degradation on ToolAlpaca after MATH500 post-training. This drop is most severe for GRPO, with a large decline of -14.97 on GLM4-9B. These results suggest that conventional RL strategies are insufficient to preserve general capabilities beyond the reward-optimized domain.

Mathematical reasoning can transfer to code generation.

In contrast, RL training on math tasks has a milder effect on coding capabilities. Several variants even improve code generation performance, indicating positive transfer from mathematical reasoning to coding, likely because both tasks rely on step-by-step reasoning and symbolic problem solving.

Method	HumanEval+	ToolAlpaca	Worst $\Delta \uparrow$
<i>Qwen3-8B</i>			
Base (Instruct)	73.58	56.53	0.00
GRPO	64.71 (−8.87)	50.72 (−5.81)	−8.87
GSPO	76.82 (+3.24)	53.41 (−3.12)	−3.12
DAPO	75.54 (+1.96)	53.09 (−3.44)	−3.44
GMPO	74.31 (+0.73)	52.90 (−3.63)	−3.63
GRPO-LoRA	74.63 (+1.05)	56.69 (+0.16)	+0.16
GCPO	77.57 (+3.99)	57.56 (+1.03)	+1.03
<i>GLM4-9B</i>			
Base (Instruct)	76.55	42.47	0.00
GRPO	76.91 (+0.36)	27.50 (−14.97)	−14.97
GSPO	81.58 (+5.03)	41.29 (−1.18)	−1.18
DAPO	81.39 (+4.84)	37.68 (−4.79)	−4.79
GMPO	79.76 (+3.21)	41.33 (−1.14)	−1.14
GRPO-LoRA	81.00 (+4.45)	43.14 (+0.67)	+0.67
GCPO	82.43 (+5.88)	43.38 (+0.91)	+0.91

Table 2: Cross-task capability retention after training on MATH500 and evaluated on HumanEval+ and ToolAlpaca benchmarks. Values in parentheses denote absolute accuracy changes relative to the base model. Worst Δ denotes the smaller change across the two evaluation tasks.

GCPO best preserves general capabilities. GCPO obtains the best Worst Δ scores on both Qwen3-8B and GLM4-9B, with gains of +1.03 and +0.91, while also improving HumanEval+ by +3.99 and +5.88, respectively. These results indicate that constraining updates away from dominant pretrained subspaces better balances task-specific RL adaptation with the preservation of general capabilities.

5.3 Optimization Dynamics of GCPO

We investigate the underlying optimization dynamics of GCPO, explicitly focusing on its ability to stabilize the training trajectory, suppress shortcut learning behaviors, and maintain hardware efficiency.

Robust Training Stability. Figure 3 illustrates the accuracy trajectories evaluated every 5 steps during training on the ToolAlpaca for Qwen3-8B. GRPO exhibits severe performance oscillations and high-variance fluctuations, whereas GCPO yields a smooth, continuously increasing accuracy curve. This is consistent with our finding that our bilateral orthogonal constraint effectively acts as a structural barrier, preventing variance-heavy gradients from intruding into the principal subspace and ensuring stable, directional policy improvement.

Stable Policy Entropy Dynamics. We further examine the policy entropy during RL post-training to characterize the exploration–exploitation behavior of different methods. A desirable optimization trajectory should gradually reduce entropy, while avoiding both entropy oscillations and premature entropy collapse (Hao et al. 2026; Cui et al. 2025). As shown in Figure 4, GRPO exhibits large entropy oscillations,

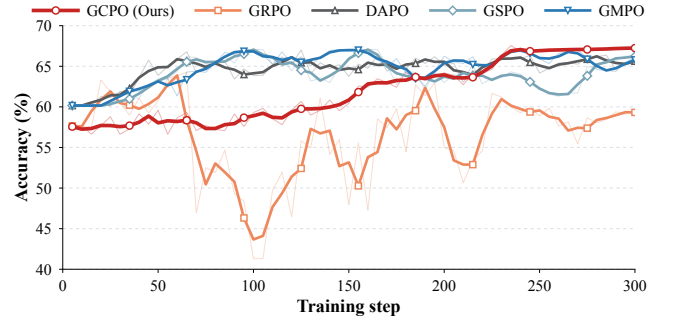


Figure 3: Training stability and accuracy trajectory on ToolAlpaca with Qwen3-8B, evaluated every 5 training steps.

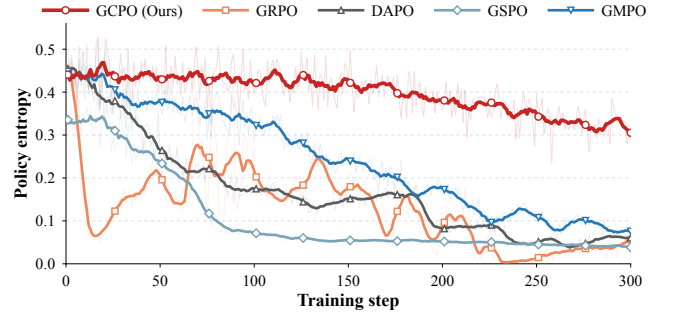


Figure 4: Policy entropy dynamics on MATH500 with GLM4-9B. GCPO maintains a smooth and gradual entropy decay, avoiding both the severe oscillations of GRPO and the premature entropy collapse in other baselines.

while most baselines exhibit rapid entropy decay, suggesting premature overconfident exploitation and reduced policy diversity. GCPO instead maintains a smooth, gradual decay, suggesting more controlled policy specialization without rapid loss of diversity.

Resistance to Response-Length Inflation. A notorious failure mode in rollout-based RL is *response-length inflation*, where the policy learns a shortcut to hack rewards by generating excessively long, redundant tokens rather than improving genuine reasoning. As shown in Figure 5, GRPO suffers from aggressive and uncontrolled output length expansion on MATH500. Conversely, GCPO substantially reduces response length, maintaining concise and stable generation lengths. We hypothesize that shielding dominant pretrained directions—including those encoding length priors—from high-variance gradients reduces the policy’s tendency to exploit verbosity as a reward-hacking shortcut.

Superior Memory Efficiency. Figure 6 compares peak GPU memory under matched GRPO training configurations. Full-parameter GRPO requires substantially more memory because optimizer states and gradients are maintained for all adapted dense weights. Both GRPO-LoRA and GCPO reduce this cost through low-rank adaptation. Importantly, GCPO achieves comparable peak memory to LoRA, showing that the bilateral geometric constraint does not sacrifice the memory efficiency of parameter-efficient training. The

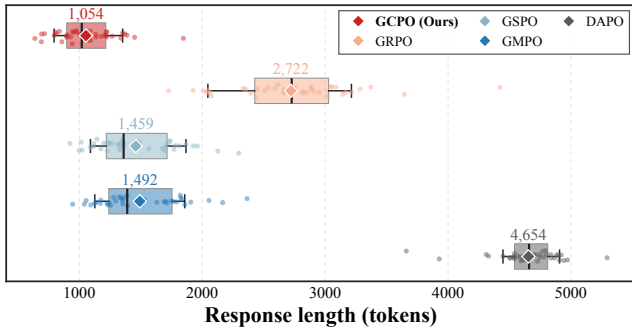


Figure 5: Response length on MATH500 with Qwen3-8B. GCPO most effectively mitigates response-length inflation.

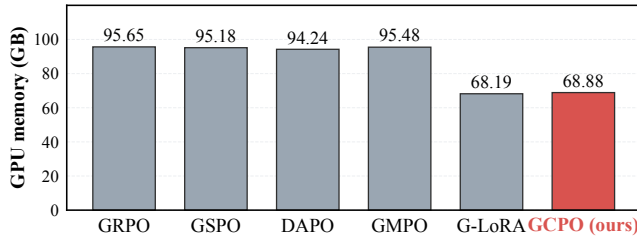


Figure 6: Peak per-GPU training memory on MATH500 with GLM4-9B, where G-LoRA abbreviates GRPO-LoRA.

principal singular vectors are frozen and require no optimizer states, while the learned update can be merged into the pretrained weights for inference.

5.4 Ablation Studies

Orthogonal Constraints Prevent Capability Collapse.

Table 3 reveals that the target subspace strongly affects optimization performance. Forcing updates into the *Principal* subspace triggers a severe performance collapse, indicating that altering dominant singular directions disrupts task-relevant capabilities. Conversely, the *Orthogonal* complement achieves peak accuracy, substantially outperforming *Random* projections. By structurally routing updates away from the core manifold, it safely resolves the conflict between aggressive task adaptation and knowledge retention.

Bilateral Projections Seal Collapse Pathways. Unilateral constraints on solely the *left* (output) or *right* (input) singular spaces yield only marginal gains over the unconstrained baseline. Geometrically, securing just one side leaves the opposite subspace vulnerable to dominant task gradients, permitting partial overlap. The *bilateral* projection is mathematically necessary to seal both pathways simultaneously, ensuring zero principal-subspace overlap and optimal accuracy.

Hard Constraints Survive Iterative Rollout Dynamics.

Both KL divergence and explicit soft orthogonality penalties ($\|\Phi_k^\top \delta W\|_F^2 + \|\delta W \Psi_k\|_F^2$) underperform our approach. Under the iterative feedback loops of RL, dominant task gradients inevitably overwhelm soft loss terms, allowing overlap to persist. By enforcing orthogonality by construction, our *Hard* constraint robustly shields the pretrained manifold.

Ablation	Variant	MATH500 Acc. $\uparrow$
Projection	w/o. Constraint	72.34
	Left-only	73.49
	Right-only	73.56
	Bilateral (Ours)	74.56
Subspace	Random	66.47
	Principal	62.59
	Orthogonal (Ours)	74.56
Constraints	Soft Loss Regularization	71.11
	KL Regularization	67.83
	Hard (Ours)	74.56

Table 3: Geometric design ablations on MATH500 benchmark with GLM4-9B, considering different projection directions, subspace selections, and constraint mechanisms.

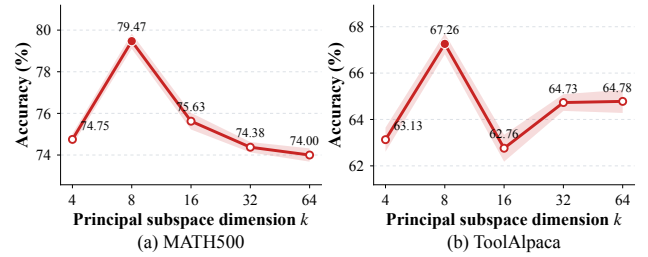


Figure 7: Effect of k on MATH500 and ToolAlpaca with Qwen3-8B, reported as mean $\pm$ standard deviation.

The Protected Dimension Balances Protection and Adaptation.

Performance varies substantially with k and peaks at $k = 8$ on both tasks (Figure 7). Smaller k provides insufficient protection, whereas larger k overly restricts the feasible update space, highlighting the importance of k in balancing subspace protection and adaptation capacity.

5.5 Limitations

Our study focuses on the parameter-space dynamics of on-policy RL, and it remains unclear whether the observed geometric patterns also extend to other post-training paradigms, such as DPO (Xu et al. 2024), KTO (Ethayarajh et al. 2024), and OPD (Kumar et al. 2025). Although principal-subspace overlap is consistently associated with performance degradation, its causal relationship with model instability, rollout behaviors such as length inflation and reward hacking is not yet fully understood. Moreover, future work should examine broader alignment objectives, model scales, and training settings, while adaptive, layer-wise selection of k , to assess the generality of this principal-subspace overlap and develop more stable and capability-preserving post-training.

6 Conclusion

We study rollout-based RL instability through the geometry of step-wise parameter updates. We identify transient principal-subspace overlap as a warning signal associated with performance degradation, with controlled interventions providing local evidence of its harmful effect. Based on this

diagnosis, we propose GCPO, which restricts effective updates to the bilateral orthogonal complements of pretrained principal subspaces. Across two backbones and three task domains, GCPO achieves the highest mean performance overall, while improving capability retention, response-length control, and policy-entropy stability. These results highlight parameter-space geometry as a useful diagnostic and design perspective for stable LLM reinforcement learning.

Acknowledgement

We gratefully acknowledge NovaCore for the valuable guidance and resources that supported this research. Additionally, We thank the open-source community for releasing code and datasets that made this research possible.

References

- Achiam, J.; Held, D.; Tamar, A.; and Abbeel, P. 2017. Constrained policy optimization. In *International conference on machine learning*, 22–31. Pmlr.
- Cai, Y.; Cao, D.; Lin, L.; Luo, C.; Xu, X.; Yang, K.; Liu, W.; Yang, S.; Zhao, T.; Sun, G.; Liu, G.; and Fang, J. 2026. Learning to Foresee: Unveiling the Unlocking Efficiency of On-Policy Distillation. *arXiv:2605.11739*.
- Cui, G.; Zhang, Y.; Chen, J.; Yuan, L.; Wang, Z.; Zuo, Y.; Li, H.; Fan, Y.; Chen, H.; Chen, W.; et al. 2025. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*.
- Dragoi, M.; Pintilie, I.; Dragomir, A.; Barbalau, A.; and Brad, F. 2026. TailLoR: Protecting Principal Components in Parameter-Efficient Continual Learning. *arXiv:2606.06494*.
- Ethayarajh, K.; Xu, W.; Muennighoff, N.; Jurafsky, D.; and Kiela, D. 2024. Kto: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*.
- Gao, L.; Schulman, J.; and Hilton, J. 2023. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, 10835–10866. PMLR.
- GLM, T.; Zeng, A.; Xu, B.; Wang, B.; Zhang, C.; Yin, D.; Rojas, D.; Feng, G.; Zhao, H.; Lai, H.; Yu, H.; Wang, H.; Sun, J.; Zhang, J.; Cheng, J.; Gui, J.; Tang, J.; Zhang, J.; Li, J.; Zhao, L.; Wu, L.; Zhong, L.; Liu, M.; Huang, M.; Zhang, P.; Zheng, Q.; Lu, R.; Duan, S.; Zhang, S.; Cao, S.; Yang, S.; Tam, W. L.; Zhao, W.; Liu, X.; Xia, X.; Zhang, X.; Gu, X.; Lv, X.; Liu, X.; Liu, X.; Yang, X.; Song, X.; Zhang, X.; An, Y.; Xu, Y.; Niu, Y.; Yang, Y.; Li, Y.; Bai, Y.; Dong, Y.; Qi, Z.; Wang, Z.; Yang, Z.; Du, Z.; Hou, Z.; and Wang, Z. 2024. ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools. *arXiv:2406.12793*.
- Golub, G. H.; Hoffman, A.; and Stewart, G. W. 1987. A generalization of the Eckart-Young-Mirsky matrix approximation theorem. *Linear Algebra and its applications*, 88: 317–327.
- Grontas, P. D.; Terpin, A.; Balta, E. C.; D’Andrea, R.; and Lygeros, J. 2025. Pinet: Optimizing hard-constrained neural networks with orthogonal projection layers. *arXiv preprint arXiv:2508.10480*.
- Hao, Z.; Wang, H.; Liu, H.; Luo, J.; Yu, J.; Dong, H.; Lin, Q.; Wang, C.; and Chen, J. 2026. Rethinking Entropy Interventions in RLVR: An Entropy Change Perspective. In Liakata, M.; Moreira, V. P.; Zhang, J.; and Jurgens, D., eds., *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 31105–31133. San Diego, California, United States: Association for Computational Linguistics. ISBN 979-8-89176-390-6.
- Harmon, J.; Hochlehnert, A.; Bethge, M.; and Prabhu, A. 2025. Mapping Post-Training Forgetting in Language Models at Scale. *arXiv preprint arXiv:2510.17776*.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W.; et al. 2022. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2): 3.
- Hübötter, J.; Lübeck, F.; Behric, L.; Baumann, A.; Bagatella, M.; Marta, D.; Hakimi, I.; Shenfeld, I.; Buening, T. K.; Guestrin, C.; and Krause, A. 2026. Reinforcement Learning via Self-Distillation. *arXiv:2601.20802*.
- Kumar, K.; Ashraf, T.; Thawakar, O.; Anwer, R. M.; Cholakal, H.; Shah, M.; Yang, M.-H.; Torr, P. H.; Khan, F. S.; and Khan, S. 2025. Llm post-training: A deep dive into reasoning large language models. *arXiv preprint arXiv:2502.21321*.
- Li, Z.; Lou, J.; Dong, F.; Fan, Z.; Ren, M.; Lin, H.; Han, X.; Zhang, D.; Sun, L.; Lu, Y.; et al. 2026. Tackling Length Inflation Without Trade-offs: Group Relative Reward Rescaling for Reinforcement Learning. *arXiv preprint arXiv:2603.10535*.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, 39578–39601.
- Liu, J.; Xia, C. S.; Wang, Y.; and Zhang, L. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35: 27730–27744.
- Schneider, J.; Schumacher, P.; Guist, S.; Chen, L.; Häufle, D.; Schölkopf, B.; and Büchler, D. 2024. Identifying Policy Gradient Subspaces. *arXiv:2401.06604*.
- Schotthöfer, S.; Zangrando, E.; Ceruti, G.; Tudisco, F.; and Kusch, J. 2024. GeoLoRA: Geometric integration for parameter efficient fine-tuning. *arXiv:2410.18720*.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*.

Shen, Z.; Li, Y.; Yin, Q.; Leong, C. T.; Wang, Z.; Chen, Y.; Han, R.; Lee, S.; and Fung, Y. R. 2026. On the Geometry of On-Policy Distillation. *arXiv:2606.07082*.

Singhal, P.; Goyal, T.; Xu, J.; and Durrett, G. 2023. A long way to go: Investigating length correlations in rlhf. *arXiv preprint arXiv:2310.03716*.

Tang, Q.; Deng, Z.; Lin, H.; Han, X.; Liang, Q.; Cao, B.; and Sun, L. 2023. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*.

Wang, H.; Li, Y.; Wang, S.; Chen, G.; and Chen, Y. 2025. MiLoRA: Harnessing Minor Singular Components for Parameter-Efficient LLM Finetuning. In Chiruzzo, L.; Ritter, A.; and Wang, L., eds., *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 4823–4836. Albuquerque, New Mexico: Association for Computational Linguistics. ISBN 979-8-89176-189-6.

Wang, X.; Chen, T.; Ge, Q.; Xia, H.; Bao, R.; Zheng, R.; Zhang, Q.; Gui, T.; and Huang, X.-J. 2023. Orthogonal subspace learning for language model continual learning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 10658–10671.

Wang, X.; Wang, S.; Liang, X.; Zhao, D.; Huang, J.; Xu, X.; Dai, B.; and Miao, Q. 2022. Deep reinforcement learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(4): 5064–5078.

Xu, S.; Fu, W.; Gao, J.; Ye, W.; Liu, W.; Mei, Z.; Wang, G.; Yu, C.; and Wu, Y. 2024. Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study. *arXiv:2404.10719*.

Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; Huang, C.; Lv, C.; et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Yu, G.; Liu, W.; Hu, Y.; Ma, H.-X.; Jiang, J.-P.; and Ye, H.-J. 2026a. Dense Supervision, Sparse Updates: On the Sparsity and Geometry of On-Policy Distillation. *arXiv:2606.13657*.

Yu, Q.; Zhang, Z.; Zhu, R.; Yuan, Y.; Zuo, X.; Yue, Y.; Dai, W.; Fan, T.; Liu, G.; Liu, L.; et al. 2026b. Dapo: An open-source llm reinforcement learning system at scale. *Advances in Neural Information Processing Systems*, 38: 113222–113244.

Zhao, Y.; Liu, Y.; Liu, J.; Chen, J.; Wu, X.; Hao, Y.; Lv, T.; Huang, S.; Cui, L.; Ye, Q.; et al. 2025. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*.

Zheng, C.; Liu, S.; Li, M.; Chen, X.-H.; Yu, B.; Gao, C.; Dang, K.; Liu, Y.; Men, R.; Yang, A.; et al. 2025. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*.

A Notations

Table 4 summarizes the main notation used in our geometric analysis and method. The superscript ℓ denotes a linear layer, and t denotes an optimization step. We omit the layer index when it is clear from context.

B Proofs

We provide proofs for the geometric properties used in the diagnostic measure and GCPO formulation. For clarity, we omit the layer index ℓ unless necessary.

Orthogonality of the Update Decomposition

Lemma B.1. *The four blocks $\delta^{(t)}W^{PP}$, $\delta^{(t)}W^{PO}$, $\delta^{(t)}W^{OP}$, and $\delta^{(t)}W^{OO}$ are mutually orthogonal under the Frobenius inner product. Consequently,*

$$\|\delta^{(t)}W\|_F^2 = E_{PP} + E_{PO} + E_{OP} + E_{OO}. \quad (9)$$

Proof. The principal and complementary projectors satisfy

$$\Pi_\Phi \Pi_\Phi^\perp = 0, \quad \Pi_\Psi \Pi_\Psi^\perp = 0. \quad (10)$$

Consider, for example, the PP and PO blocks. Their Frobenius inner product is

$$\begin{aligned} & \left\langle \Pi_\Phi \delta^{(t)}W \Pi_\Psi, \Pi_\Phi \delta^{(t)}W \Pi_\Psi^\perp \right\rangle_F \\ &= \text{tr} \left(\Pi_\Psi \delta^{(t)}W^\top \Pi_\Phi \delta^{(t)}W \Pi_\Psi^\perp \right) = 0, \end{aligned} \quad (11)$$

where the last equality follows from cyclic invariance of the trace and $\Pi_\Psi^\perp \Pi_\Psi = 0$. The remaining pairs follow analogously from orthogonality of the left or right projectors. The squared norm of their sum is therefore the sum of their squared norms.

It follows immediately that the fraction of energy overlapping at least one principal subspace is

$$O_t = \frac{E_{PP} + E_{PO} + E_{OP}}{E_{\text{total}}} = 1 - \frac{E_{OO}}{E_{\text{total}}}. \quad (12)$$

Expected Overlap of an Isotropic Update

Proposition B.2. *Let $\delta W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ have a uniformly random direction under the Frobenius norm. Then its expected principal-subspace overlap is*

$$\mathbb{E}[O] = 1 - \frac{(d_{\text{out}} - k)(d_{\text{in}} - k)}{d_{\text{out}} d_{\text{in}}}. \quad (13)$$

Proof. Let

$$z = \text{vec}(\delta W) \in \mathbb{R}^D, \quad D = d_{\text{out}} d_{\text{in}}. \quad (14)$$

Using the vectorization identity, the doubly orthogonal component can be written as

$$\text{vec}(\Pi_\Phi^\perp \delta W \Pi_\Psi^\perp) = Qz, \quad Q = \Pi_\Psi^\perp \otimes \Pi_\Phi^\perp. \quad (15)$$

The matrix Q is an orthogonal projector with rank

$$\text{rank}(Q) = (d_{\text{out}} - k)(d_{\text{in}} - k). \quad (16)$$

For a uniformly random direction,

$$\mathbb{E} \left[\frac{zz^\top}{\|z\|_2^2} \right] = \frac{I_D}{D}. \quad (17)$$

Therefore,

$$\mathbb{E} \left[\frac{\|Qz\|_2^2}{\|z\|_2^2} \right] = \text{tr} \left(Q \mathbb{E} \left[\frac{zz^\top}{\|z\|_2^2} \right] \right) \quad (18)$$

$$= \frac{\text{rank}(Q)}{D} \quad (19)$$

$$= \frac{(d_{\text{out}} - k)(d_{\text{in}} - k)}{d_{\text{out}} d_{\text{in}}}. \quad (20)$$

Since $O = 1 - \|Qz\|_2^2 / \|z\|_2^2$, the stated result follows.

This result gives the dimensionality-dependent null value O_{null} used in the excess-overlap statistic.

Exact Feasibility and Subspace Preservation

Proposition B.3. *Suppose the cumulative adaptation of layer ℓ is parameterized as*

$$\Delta W_t^{(\ell)} = s \Pi_\Phi^{(\ell)\perp} L_t^{(\ell)} R_t^{(\ell)} \Pi_\Psi^{(\ell)\perp}. \quad (21)$$

Then both the cumulative adaptation and every realized update satisfy the bilateral constraints:

$$\Phi_k^{(\ell)\top} \Delta W_t^{(\ell)} = 0, \quad \Delta W_t^{(\ell)} \Psi_k^{(\ell)} = 0, \quad (22)$$

and

$$\Phi_k^{(\ell)\top} \delta^{(t)}W^{(\ell)} = 0, \quad \delta^{(t)}W^{(\ell)} \Psi_k^{(\ell)} = 0. \quad (23)$$

Proof. By construction,

$$\Phi_k^{(\ell)\top} \Pi_\Phi^{(\ell)\perp} = 0, \quad \Pi_\Psi^{(\ell)\perp} \Psi_k^{(\ell)} = 0. \quad (24)$$

Hence,

$$\Phi_k^{(\ell)\top} \Delta W_t^{(\ell)} = s \Phi_k^{(\ell)\top} \Pi_\Phi^{(\ell)\perp} L_t^{(\ell)} R_t^{(\ell)} \Pi_\Psi^{(\ell)\perp} = 0, \quad (25)$$

$$\Delta W_t^{(\ell)} \Psi_k^{(\ell)} = s \Pi_\Phi^{(\ell)\perp} L_t^{(\ell)} R_t^{(\ell)} \Pi_\Psi^{(\ell)\perp} \Psi_k^{(\ell)} = 0. \quad (26)$$

The realized update is

$$\delta^{(t)}W^{(\ell)} = \Delta W_t^{(\ell)} - \Delta W_{t-1}^{(\ell)}. \quad (27)$$

Because the constraints are linear and both cumulative adaptations satisfy them, their difference satisfies them as well.

Corollary B.4. *For every $x \in \text{span}(\Psi_k^{(\ell)})$ and $y \in \text{span}(\Phi_k^{(\ell)})$,*

$$W_t^{(\ell)} x = W_{\text{ref}}^{(\ell)} x, \quad y^\top W_t^{(\ell)} = y^\top W_{\text{ref}}^{(\ell)}. \quad (28)$$

Proof. Write $x = \Psi_k^{(\ell)} a$ and $y = \Phi_k^{(\ell)} b$. The bilateral constraints imply $\Delta W_t^{(\ell)} x = 0$ and $y^\top \Delta W_t^{(\ell)} = 0$. Substituting $W_t^{(\ell)} = W_{\text{ref}}^{(\ell)} + \Delta W_t^{(\ell)}$ proves the result.

The corollary is a layer-level algebraic guarantee for the selected subspaces; it does not imply unconditional preservation of all model-level capabilities.

Table 4: Summary of the main notation.

Notation	Definition
θ_0, θ_t	Initial policy parameters and policy parameters after optimization step t .
$\pi_{\theta_t}, \pi_{\text{ref}}$	Current policy and fixed reference policy.
$\mathcal{M}$	Set of linear layers adapted during RL post-training.
$W_{\text{ref}}^{(\ell)}, W_t^{(\ell)}$	Pretrained and adapted weight matrices of layer ℓ .
$W_{\text{ref}}^{(\ell)} = \Phi^{(\ell)} \Sigma^{(\ell)} \Psi^{(\ell)\top}$	Singular value decomposition of the pretrained weight.
$\Phi_k^{(\ell)}, \Psi_k^{(\ell)}$	Top- k left and right singular vectors, defining the principal output and input subspaces, respectively.
$\Pi_{\Phi}^{(\ell)}, \Pi_{\Psi}^{(\ell)}$	Projectors onto the principal subspaces: $\Pi_{\Phi}^{(\ell)} = \Phi_k^{(\ell)} \Phi_k^{(\ell)\top}$ and $\Pi_{\Psi}^{(\ell)} = \Psi_k^{(\ell)} \Psi_k^{(\ell)\top}$.
$\Pi_{\Phi}^{(\ell)\perp}, \Pi_{\Psi}^{(\ell)\perp}$	Projectors onto the corresponding orthogonal complements.
$\Delta W_t^{(\ell)} = W_t^{(\ell)} - W_{\text{ref}}^{(\ell)}$	Cumulative adaptation relative to the pretrained weight.
$\delta^{(t)} W^{(\ell)} = W_t^{(\ell)} - W_{t-1}^{(\ell)}$	Realized update at optimization step t .
$\delta^{(t)} W_{ij}^{(\ell)}$	Update block obtained by projecting its left and right sides onto $i, j \in \{P, O\}$, where P and O denote the principal and orthogonal-complement subspaces.
$E_{ij}^{(t,\ell)}$	Squared Frobenius energy of block ij : $\ \delta^{(t)} W_{ij}^{(\ell)}\ _F^2$.
O_t	Fraction of update energy overlapping at least one principal subspace.
O_{null}	Expected overlap of an isotropic update under the same matrix dimensions and protected rank.
$O_t^{\text{excess}} = O_t - O_{\text{null}}$	Dimension-corrected principal-subspace overlap.
$L^{(\ell)}, R^{(\ell)}$	Trainable low-rank factors used by GCPO.
r, k, s	Adaptation rank, protected principal rank, and adaptation scaling coefficient, respectively.
$\ \cdot\ _F$	Frobenius norm.

Dimension and Expressivity of the Feasible Space

Proposition B.5. *Define the bilateral feasible space*

$$\mathcal{S} = \{A : \Phi_k^\top A = 0, A \Psi_k = 0\}. \quad (29)$$

Then

$$\mathcal{S} = \{\Pi_{\Phi}^\perp Z \Pi_{\Psi}^\perp : Z \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}\}, \quad (30)$$

and

$$\dim(\mathcal{S}) = (d_{\text{out}} - k)(d_{\text{in}} - k). \quad (31)$$

Moreover, every matrix in $\mathcal{S}$ with rank at most r can be represented by the GCPO low-rank parameterization.

Proof. If $A \in \mathcal{S}$, then its columns lie in $\text{span}(\Phi_k)^\perp$, implying $\Pi_{\Phi}^\perp A = A$. Similarly, $A \Psi_k = 0$ implies $A \Pi_{\Psi}^\perp = A$. Thus,

$$A = \Pi_{\Phi}^\perp A \Pi_{\Psi}^\perp. \quad (32)$$

Let $\Phi_\perp$ and $\Psi_\perp$ be orthonormal bases for the corresponding complementary subspaces. Every feasible matrix has a unique representation

$$A = \Phi_\perp B \Psi_\perp^\top, \quad B \in \mathbb{R}^{(d_{\text{out}} - k) \times (d_{\text{in}} - k)}. \quad (33)$$

Therefore, the dimension of $\mathcal{S}$ is $(d_{\text{out}} - k)(d_{\text{in}} - k)$.

If $\text{rank}(A) \leq r$, then $\text{rank}(B) \leq r$, so B admits a factorization $B = UV$ with at most r latent dimensions. Taking

$$L = \Phi_\perp U, \quad R = V \Psi_\perp^\top \quad (34)$$

gives

$$A = \Pi_{\Phi}^\perp L R \Pi_{\Psi}^\perp, \quad (35)$$

which is exactly the GCPO parameterization up to its scaling coefficient.

C Method Details

Global Overlap Aggregation and Robustness

Global aggregation. For each adapted matrix $\ell \in \mathcal{M}$, we first compute the layer-wise dimension-corrected overlap

$$O_{t,\ell}^{\text{excess}} = \left(1 - \frac{E_{OO}^{(t,\ell)}}{E_{\text{total}}^{(t,\ell)}}\right) - \left[1 - \frac{(d_{\text{out}}^{(\ell)} - k_\ell)(d_{\text{in}}^{(\ell)} - k_\ell)}{d_{\text{out}}^{(\ell)} d_{\text{in}}^{(\ell)}}\right]. \quad (36)$$

The global statistic shown in Figure 1 is the unweighted macro-average

$$O_{t,\text{global}}^{\text{excess}} = \frac{1}{|\mathcal{M}|} \sum_{\ell \in \mathcal{M}} O_{t,\ell}^{\text{excess}}. \quad (37)$$

Normalization and null correction are therefore performed within each matrix before aggregation. Consequently, each adapted matrix contributes equally, preventing matrices with larger dimensions or update norms from mechanically dominating the global statistic. The displayed curve applies a five-step moving average only after this layer-wise aggregation.

Algorithm Details

Algorithm 1 summarizes the implementation of GCPO. The principal subspaces are computed once before training, while the projected low-rank factors are used for both policy optimization and rollout generation.

D Experimental Details

Datasets and Splits

Table 5 summarizes the datasets used for RL training and held-out evaluation. MATH500 and HumanEval+ are established benchmarks for mathematical reasoning and code

Algorithm 1: Geometrically Constrained Policy Optimization

Input: Frozen policy θ_0 and adapted layers $\mathcal{M}$
Parameter: Protected rank k , adaptation rank r , scaling α , and training steps T
Output: Trained factors $\{L^{(\ell)}, R^{(\ell)}\}_{\ell \in \mathcal{M}}$

- 1: Set $s \leftarrow \alpha/r$.
- 2: **for** each layer $\ell \in \mathcal{M}$ **do**
- 3: Compute the top- k singular vectors $\Phi_k^{(\ell)}$ and $\Psi_k^{(\ell)}$ of $W_{\text{ref}}^{(\ell)}$.
- 4: Initialize $L^{(\ell)}$ to zero and $R^{(\ell)}$ with the standard LoRA initialization.
- 5: **end for**
- 6: **for** $t = 1, \dots, T$ **do**
- 7: **for** each layer $\ell \in \mathcal{M}$ **do**
- 8: $\bar{L}^{(\ell)} \leftarrow L^{(\ell)} - \Phi_k^{(\ell)} (\Phi_k^{(\ell)\top} L^{(\ell)})$.
- 9: $\bar{R}^{(\ell)} \leftarrow R^{(\ell)} - (R^{(\ell)} \Psi_k^{(\ell)}) \Psi_k^{(\ell)\top}$.
- 10: Set $W_t^{(\ell)} \leftarrow W_{\text{ref}}^{(\ell)} + s \bar{L}^{(\ell)} \bar{R}^{(\ell)}$.
- 11: **end for**
- 12: Synchronize the projected factors to the rollout engine.
- 13: Generate rollouts and compute the policy-optimization loss.
- 14: Update the raw factors $\{L^{(\ell)}, R^{(\ell)}\}_{\ell \in \mathcal{M}}$.
- 15: **end for**
- 16: **return** $\{L^{(\ell)}, R^{(\ell)}\}_{\ell \in \mathcal{M}}$.

Table 5: Dataset splits used for RL training and held-out evaluation.

Task	Total	Train	Test	Protocol
MATH500	500	450	50	9:1, seed 42.
HumanEval+	164	147	17	9:1, seed 42.
ToolAlpaca	4,114	4,046	68	407:10 families

generation, respectively. Following the common practice of constructing fixed train–test splits for controlled RL evaluation (Hübotter et al. 2026), we build task-specific splits from their official evaluation sets and keep them identical across all methods. Consequently, the corresponding results measure performance on our held-out subsets and should not be interpreted as official full-benchmark scores.

For MATH500 (Lightman et al. 2024) and HumanEval+ (Liu et al. 2023), no problem identifier appears in both the training and evaluation splits. Each HumanEval+ problem retains its complete EvalPlus functional-test suite; tests associated with the same programming problem are never divided between the two splits.

For ToolAlpaca, constructed from ToolAlpaca (Tang et al. 2023), the held-out set contains examples from 10 API families that do not appear during training: Axolotl, Auth0, A Biblia Digital, Apache Superset, Améthyste, Abstract Public Holidays, AbuseIPDB, 1Forge, Lob.com, and AniAPI. This

Table 6: Shared training, rollout, and evaluation settings.

Configuration	Setting
Train batch size	32 prompts per optimization step
Per-GPU micro-batch size	1
Optimizer	AdamW; gradient clipping at 1.0
Data order	Shuffled during training
Training precision	bfloat16 with Fully Sharded Data Parallel (FSDP)
Rollout engine	vLLM with tensor parallelism of 2
Maximum prompt length	2,048 tokens
Maximum response length	8,192 tokens
Training decoding	Temperature 1.0, top- $p = 1.0$, and top- $k = -1$
Evaluation decoding	Temperature 0.6 and top- $p = 0.95$
Evaluation samples	16 sampled responses per held-out example
Evaluation metric	Majority@16 with 1,000 bootstrap resamples
Evaluation interval	Every 5 optimization steps
Checkpoint interval	Every 50 steps by default; every 100 steps for DAPO
Checkpoint retention	At most one checkpoint retained per run
Chat template	Model-specific template; thinking mode is disabled for Qwen3
Independent runs	3 random seeds per main configuration
Result reporting	Mean $\pm$ standard deviation over the 3 independent runs

protocol evaluates generalization to previously unseen tool families rather than to new examples from APIs observed during training.

The MATH500 and HumanEval+ splits are generated once with seed 42 and kept fixed across all methods, backbone models, and training runs. This data-splitting seed is independent of the three random seeds used for repeated training. Therefore, the reported variation across runs reflects training and sampling stochasticity, rather than changes in the train–test partition.

Training and Evaluation Settings

Table 6 summarizes the settings shared across tasks and methods unless overridden by a method-specific configuration. All main model–task–method configurations are independently trained with three random seeds. For each configuration, we report the mean and standard deviation of the three resulting task-level scores.

For each training seed, we generate 16 responses per held-out example and report majority@16 to reduce sensitivity to individual stochastic generations and evaluate the model’s consistency across multiple samples. We estimate majority@16 using 1,000 bootstrap trials, each of which resamples 16 responses with replacement and scores the majority-voted answer. Correctness is averaged over bootstrap trials and examples to obtain one score per training run, and final results are reported as the mean and standard deviation over three independent runs.

Table 7: Effective training horizons. All runs are configured for at most 300 steps, with a batch size of 32, `drop_last=True`, and at most 30 epochs.

Task	Train size	Steps / epoch	30-epoch cap	Effective steps
MATH500	450	14	420	300
HumanEval+	147	4	120	120
ToolAlpaca	4,046	126	3,780	300

Table 8: Method-specific rollout and optimization hyperparameters.

Method	Roll-outs	Learning rate	Principal settings
GRPO	16	1×10^{-5}	Mini-batch size 32; clipping ratio 0.2.
GSPO	16	1×10^{-6}	Lower and upper clipping thresholds of 3×10^{-4} and 4×10^{-4} .
GMPO	16	1×10^{-6}	lower and upper clipping thresholds of 0.4.
DAPO	16	1×10^{-6}	Train batch size 32; generation batch size 64.

Effective Training Horizons. The common training loop uses `drop_last=True` and a default limit of 30 epochs. As a result, the effective number of optimization steps depends on the size of the corresponding training split. Table 7 summarizes the resulting training horizons.

MATH500 and ToolAlpaca can reach the configured limit of 300 optimization steps. HumanEval+ contains 147 training problems, yielding four complete batches per epoch and therefore at most 120 steps under the default 30-epoch limit. For cross-task evaluation, we use the same task-specific checkpoint for every method and seed: step 250 for MATH500, step 100 for HumanEval+, and step 150 for ToolAlpaca. We do not select a separate best-validation checkpoint for each seed.

Method-specific Hyperparameters

We retain method-specific optimization settings rather than forcing all algorithms to share a common rollout count, learning rate, or clipping rule. Table 8 lists the principal hyperparameters. The same configuration is used across the three independent runs of each method.

The parameter-efficient configurations are summarized in Table 9. LoRA and GCPO adapt all linear layers. For matched comparisons, the full-parameter, LoRA, and GCPO variants use identical data splits, batch sizes, rollout settings, and evaluation protocols; they differ only in the parameterization of the policy update.

The main GCPO configuration uses scaling parameter $\alpha = 16$, and protects the top $k = 8$ left and right singular directions of each adapted weight matrix. The ablation studies additionally consider $k \in \{4, 8, 16, 32, 64\}$ and adaptation

Table 9: Parameter-efficient adaptation settings used in the main experiments.

Variant	Modules	Rank r	α	Protected rank k
LoRA	All linear	32	16	—
GCPO	All linear	32	16	8

Table 10: Task-specific response extraction and scoring.

Task	Extraction	Correctness
MATH500	Final <code>\boxed{}</code> expression in the last 100 characters	Normalized string matching with Math-Verify equivalence checking
HumanEval+	Longest Markdown code block	Pass all EvalPlus functional tests within 5 seconds
ToolAlpaca	Action names and input JSON	Unordered action-name match and merged argument-JSON match; no tool execution

ranks $r \in \{16, 32, 64\}$.

Reward and Evaluation Protocols

All three tasks use binary response-level rewards. Table 10 summarizes the task-specific prediction extraction and correctness criteria. A response receives a binary reward of 1 if it satisfies the task-specific criterion and 0 otherwise.

For MATH500, the model is prompted to provide its final answer inside `\boxed{}`. For HumanEval+, each problem retains its full functional-test suite, and a generated program is considered correct only when it passes every test. The 5-second execution limit is used by the evaluator, regardless of the shorter time limit stated in the textual prompt.

The ToolAlpaca evaluator measures static structural correctness rather than actual API execution. It does not evaluate the order of multiple actions, and arguments from multiple predicted actions are merged before comparison. Accordingly, we describe this metric as action-name and argument matching rather than execution correctness.

For every training seed, each held-out example is evaluated using 16 sampled responses under the decoding configuration in Table 6. The resulting responses are aggregated into a single task-level score for that seed. Final results are reported as the mean and standard deviation across the three independently trained seeds.

Computational Resources

All experiments were conducted on a single compute node equipped with four NVIDIA A100 GPUs. Table 11 summarizes the hardware configuration and distributed training setup. The 48-hour allocation denotes the maximum wall-clock budget assigned to each job, rather than the actual runtime of every training run.

For the matched GRPO comparison, full-parameter GRPO, GRPO-LoRA, and GCPO use identical data splits, batch sizes, rollout configurations, and evaluation protocols.

Table 11: Computational resources and distributed training configuration used in our experiments.

Item	Configuration
Compute node	Single node
GPU	4× NVIDIA A100 80 GB
CPU	32 CPU cores
Job time limit	48 hours per submitted job
Training precision	bfloat16
Distributed training	Fully Sharded Data Parallel (FSDP)
Rollout engine	vLLM
Rollout parallelism	Tensor parallelism with $TP = 2$
Micro-batch size	1 sample per GPU

They differ only in the parameterization and geometric constraint of the policy update.

The effective number of optimization steps additionally depends on the dataset size because the common trainer uses `drop_last=True` and terminates after 30 epochs. Table 7 reports the resulting training horizon for each task.

For MATH500 and ToolAlpaca, the epoch limit permits the trainer to reach the configured maximum of 300 steps. For HumanEval+, the 147-example training split produces only four complete batches per epoch, resulting in at most 120 optimization steps. Since checkpoints are saved every 50 steps, we use the step-100 checkpoint for cross-task evaluation. DAPO uses a separate epoch budget of 3,000 and is therefore not subject to the default 120-step limit on HumanEval+.

E Additional Experiments

Additional Overlap Trajectories

Figure 8 complements the representative cases in Figure 1 with additional trajectories on HumanEval+ and ToolAlpaca. We use the same excess-overlap measure, 5-step moving average, and validation protocol as in Section 3.2. Elevated overlap again tends to accompany or precede validation degradation, although its timing and magnitude vary across tasks. To avoid redundant per-seed plots, we show one fixed-seed trajectory for each remaining task.

Additional Controlled Interventions

We repeat the layer-wise norm-matched intervention on three additional model–task–checkpoint configurations. As shown in Figure 9 and 10, increasing the principal-overlapping component consistently produces a dose-dependent accuracy drop across settings. These results support the robustness of the local intervention effect beyond the main configuration.

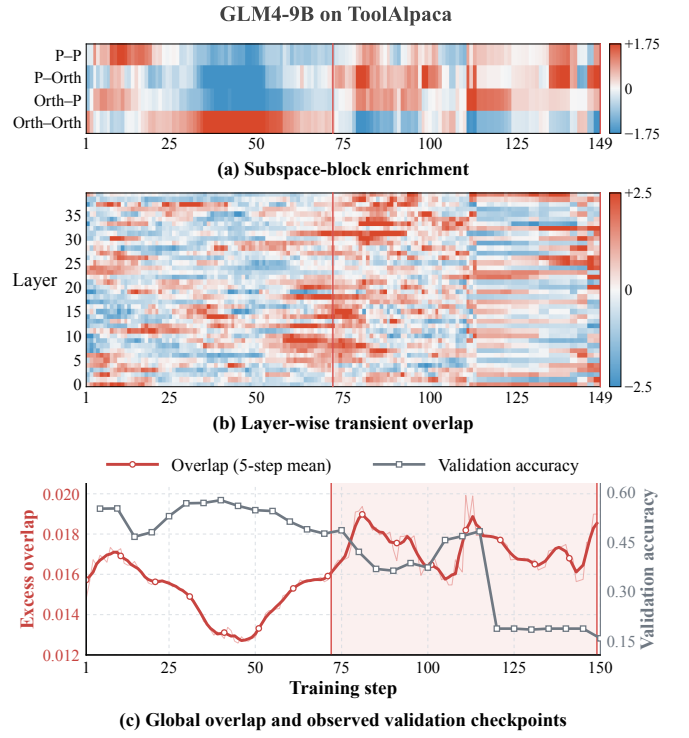
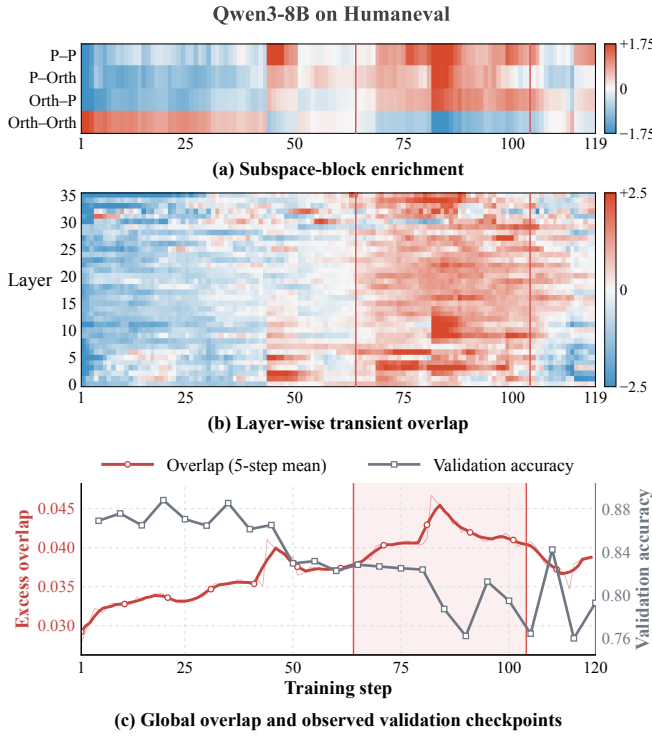


Figure 8: Stepwise update overlap and validation performance. The red curve is the 5-step moving average of excess principal-subspace overlap; the blue curve is validation accuracy. In both runs, episodes of elevated overlap accompany subsequent gradual degradation. This observation is correlational.

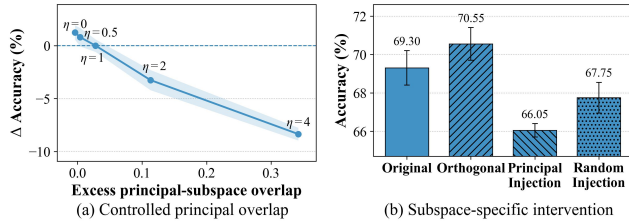


Figure 9: Additional controlled intervention on the step-125 GRPO update of Qwen3-8B on MATH500. (a) Increasing the principal-overlapping component under layer-wise norm matching produces a dose-dependent accuracy drop. (b) Orthogonalization improves accuracy, whereas matched principal-subspace injection is substantially more harmful than a random-subspace intervention.

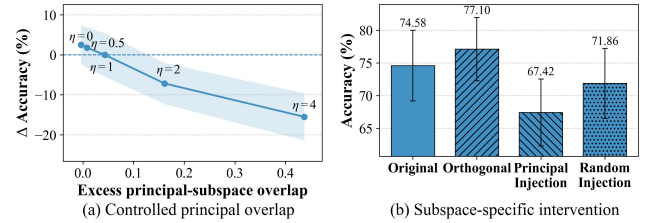


Figure 10: Additional controlled intervention on the step-70 GRPO update of GLM on Humaneval. (a) Increasing the principal-overlapping component under layer-wise norm matching produces a dose-dependent accuracy drop. (b) Orthogonalization improves accuracy, whereas matched principal-subspace injection is substantially more harmful than a random-subspace intervention.